

SQL i zapytania SQL SELECT – cz. II

SQL and SQL SELECT queries – part II

dr hab. inż. Maciej Grzenda

M.Grzenda@mini.pw.edu.pl

<http://www.mini.pw.edu.pl/~grzendam>

Politechnika Warszawska
Wydział Matematyki i Nauk Informacyjnych



**European
Funds**

Knowledge Education Development

**Warsaw University
of Technology**

European Union
European Social Fund



MSc program in Data Science has been developed
as a part of task 10 of the project
„NERW PW. Science - Education - Development - Cooperation”
co-funded by European Union from European Social Fund.

Challenges related to the use of data present in databases

- Until now, we have used SQL to query raw data from one or more tables.
- But the question remains:
 - Can we go beyond simply finding records that match certain criteria? For example, can we calculate the total number and value of orders made from different countries?

The answer: using Structured Query Language (SQL), you can also retrieve aggregated data from a database. This is another reason why knowing **SQL is a key database skill** — it helps **analyse trends and measure business performance**.

Objectives

- SQL is a key language in databases domain
- SQL SELECT queries typically go beyond joining data with JOIN and filtering with WHERE
- The queries frequently:
 - Aggregate the data e.g. by calculating sums
 - Look for records in one table for which no matching records in other tables exist
 - Combine aggregation with filtering based on the values of aggregates e.g. to find customers with total order value exceeding some threshold value

GROUP BY phrase

- Allows to group the records with the same values of some columns
- Can be used with SELECT statements referring to one or many tables
- Is typically used with aggregate functions such as SUM() or MIN()

Aggregate functions

Category	Description
COUNT(*)	Calculates the number of records
SUM(expression)	Calculates the sum of values of a numerical expression calculated for individual input rows
AVG(expression)	Calculates the average value of a numerical expression calculated for individual input rows
MIN(expression)	Calculates the lowest value of an expression. Values are compared based on their type – in the same way they are compared to be sorted.
MAX(expression)	Calculates the largest value of an expression. Values are compared based on their type – in the same way they are compared to be sorted.

Note that an expression can be just a column name e.g. `NetValue` or is based on column value e.g. `SUM (NetValue*1.23)` can be calculated

Trivial example to start from

```
SELECT COUNT(*) AS order_count,  
SUM(order_value) AS  
total_order_value  
FROM Orders  
WHERE ShipCountry='France'
```

order_count	total_order_value
42	37890

This query will calculate ... the total number of orders (i.e. order records) and their total value. First records matching filtering condition will be found. Next one row with results will be returned as the result set of the query.

GROUP BY - example

- Calculate the number of orders and their total value for each customer

- ```
SELECT Customer_ID,
COUNT(*) AS order_count,
SUM(order_value) AS
total_order_value
FROM Orders
GROUP BY Customer_ID
```



**All the aggregate function values will be calculated for every group of records with the same Customer\_ID independently**



## Query interpretation – step I

First, list of records matching WHERE part is obtained

| Customer_ID | Order_Date | Order_Value | Delivery_City | ...  |
|-------------|------------|-------------|---------------|------|
| MAIN        | 01/02/2002 | 200.00      | Warszawa      | ...  |
| MAIN        | 01/02/2013 | 100.00      | Poznań        |      |
| PW          | 03/05/2001 | 20          | Warszawa      | ...  |
| PW          | 03/06/1987 | 80          | Warszawa      | ...  |
| MAIN        | 14/11/2001 | 20          | Warszawa      | .... |

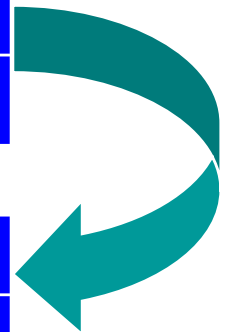
**To answer a query with GROUP BY unique list of column values is retrieved (here: unique list of CUSTOMER\_ID)**

## Query interpretation – step II

For every group i.e. every value of grouping expression, aggregate functions are calculated. Here the output for Customer\_ID='MAIN' is shown

| Customer_ID | Order_Date | Order_Value | Delivery_City | ...  |
|-------------|------------|-------------|---------------|------|
| MAIN        | 01/02/2002 | 200.00      | Warszawa      | ...  |
| MAIN        | 01/02/2013 | 100.00      | Poznań        |      |
| MAIN        | 14/11/2001 | 20          | Warszawa      | .... |

| Customer_ID | Order_Count | Total_Order_Value |
|-------------|-------------|-------------------|
| MAIN        | 3           | 320.00            |



**Here is how the output row for Customer\_ID='MAIN' is obtained. Similar procedure is performed for every Customer\_ID value present in the source data**

## Query interpretation – step III

Finally the result set of the query looks as follows:

```
SELECT Customer_ID, COUNT(*) AS
order_count, SUM(order_value) AS
total_order_value
FROM Orders
GROUP BY Customer_ID
```

| Customer_ID | Order_Count | Total_Order_Value |
|-------------|-------------|-------------------|
| PW          | 2           | 100.00            |
| MAIN        | 3           | 320.00            |

## GROUP BY - discussion

- Only columns used in GROUP BY phrase can be used in SELECT list, thus one cannot use e.g. a query:

```
SELECT customer_id, delivery_city,... FROM
...
GROUP BY customer_id
```

- Why? Because there are potentially many delivery\_cities for the same customer\_id. Which one should be displayed then if there is one record in the output per one customer?
- GROUP BY allows to reduce resource use both on server side and client side, whenever summary values instead of detailed records are required

## GROUP BY – multiple columns

- Calculate the number of orders and their total value for every customer and delivery city
- ```
SELECT  Customer_ID, Delivery_City ,  
        COUNT(*) AS order_count,  
        SUM(order_value) AS total_order_value  
FROM Orders  
GROUP BY Customer_ID, Delivery_City
```

All the aggregate function values will be calculated for each group of records with the same Customer_ID and Delivery_City independently

Query interpretation – step II

For each group, the aggregate functions are calculated.

Customer_ID	Order_Date	Order_Value	Delivery_City	...
MAIN	01/02/2022	200.00	Warszawa	...
MAIN	01/02/2025	100.00	Poznań	
MAIN	14/11/2024	20	Warszawa	

Customer_ID	Delivery_City	Order_Count	Total_Order_Value
MAIN	Poznań	1	100.00
MAIN	Warszawa	2	220.00

Here is how the output row for Customer_ID='MAIN' is obtained. Similar procedure is performed for each combination of Customer_ID and Delivery_City present in the source data

HAVING phrase

- Allow to impose logical condition on the result of aggregate function

- ```
SELECT Customer_ID, Delivery_City ,
 COUNT(*) AS order_count,
 SUM(order_value) AS total_order_value
FROM Orders
GROUP BY Customer_ID, Delivery_City
HAVING SUM(order_value)>150
```

**Only these rows of the final result set that fulfil the HAVING condition will be placed in the result set**

# Query interpretation – HAVING

For every group, the functions are calculated.

| Customer_ID | Order_Date | Order_Value | Delivery_City | ...  |
|-------------|------------|-------------|---------------|------|
| MAIN        | 01/02/2022 | 200.00      | Warszawa      | ...  |
| MAIN        | 01/02/2025 | 100.00      | Poznań        |      |
| MAIN        | 14/11/2024 | 20          | Warszawa      | .... |

| Customer_ID | Delivery_City | Order_Count | Total_Order_Value |
|-------------|---------------|-------------|-------------------|
| MAIN        | Poznań        | 1           | 100.00            |
| MAIN        | Warszawa      | 2           | 220.00            |

**Only the records that are compliant with HAVING condition are left in the final result set**



## NULL values and comparisons

- How to compare against NULL?

```
WHERE Column_Name is [NOT]
NULL ...
```

- Be careful with MIN(), MAX(), SUM() – they can produce not obvious results whenever NULL values in the aggregated expressions are used

# SQL and sample data analysis I

```
library(gdata)
library(sqldf)
stats <-
 read.csv(
 inputFileName, sep = ';', dec = ',', stringsAsFactors = F
)
delaysPreprocessed <- sqldf('select * from stats where
delay<1000000 order by delay desc')
delaysPreprocessed$arrivalHour<-
as.numeric(substr(delaysPreprocessed$arrivalTime, regexpr(':',
delaysPreprocessed$arrivalTime)-
2, regexpr(':', delaysPreprocessed$arrivalTime)-1))

delays<-sqldf('select * from delaysPreprocessed where
delay<=60*20 and delay>=-60*20
```

...

**A part of modern analysis is based on mixing matrix-like and file-based operations with the use of SQL. The sample code shown here is developed in R.**

## SQL and sample data analysis II

```
positiveDelaysPerStop<-sqldf('select
 sum(abs(delayRoundToMinutes))
 totalRoundedDelayInMinutes,
 avg(abs(delayRoundToMinutes)) as
 averageRoundedDelayInMinutes,
 poleStop,pole, stop from delays
 where delayRoundToMinutes>0
 group by poleStop,pole,stop order by
 averageRoundedDelayInMinutes desc')
and arrivalHour>=4 and arrivalHour<=23
order by delay desc')
```

In this case, SQL code is executed on top of in-memory data imported from a text file.

# Views

- View can be treated as a saved query

- Example:

```
- CREATE VIEW TotalSale AS SELECT Customer_ID,
COUNT(*) AS order_count, SUM(order_value) AS
total_order_value
FROM Orders
GROUP BY Customer_ID
- SELECT * FROM TotalSale WHERE
total_order_value<250
- SELECT * FROM TotalSale WHERE
Customer_ID='ABC'
```

**After defining a view, it can be used in queries just as a standard table.  
However, the performance may be degraded.**

# Views - guidelines

- Views, by default, do not store data. Every time a query is submitted referring to a view, the data from underlying tables are used by DBMS to answer the query
- Can be created to restrict access e.g. a user can access total sales, but not detailed information on each order
- Views should not be defined whenever there is a need to use ad hoc queries
- Views provide means for system integration e.g. remote applications can access a view without knowing the physical model of the database in which view has been defined

# Any problems here?

## Business request:

- calculate total quantity of products ordered in 1997
- answer:

```
select SUM(quantity) as
totalQuantity
from [order details] od
join orders o on
o.OrderID=od.OrderID
where
year(orderDate)=1997
```

| ProductName      | Unit             |
|------------------|------------------|
| Queso Cabrales   | 1 kg pkg.        |
| Konbu            | 2 kg box         |
| Tofu             | 40 - 100 g pkgs. |
| Alice Mutton     | 20 - 1 kg tins   |
| Carnarvon Tigers | 16 kg pkg.       |

# Summary

- SQL is a key language in databases domain
- The complexity of SQL queries goes beyond trivial reading raw data from a single table
- Data aggregation is frequently used especially, when trends rather than raw facts have to be revealed
- Further details on SQL queries including other SQL capabilities will be provided during the laboratories

## Questions?

I am looking forward to your questions during the lectures (including now) and during my office hours

For details on my office hours, please check my website:

[https://pages.mini.pw.edu.pl/~grzendam/eng/dydaktyka\\_eng.html](https://pages.mini.pw.edu.pl/~grzendam/eng/dydaktyka_eng.html)







**European  
Funds**

Knowledge Education Development

**Warsaw University  
of Technology**

**European Union**  
European Social Fund



MSc program in Data Science has been developed  
as a part of task 10 of the project  
„NERW PW. Science - Education - Development - Cooperation”  
co-funded by European Union from European Social Fund.